VENKATA NAGA KIRAN KATARI

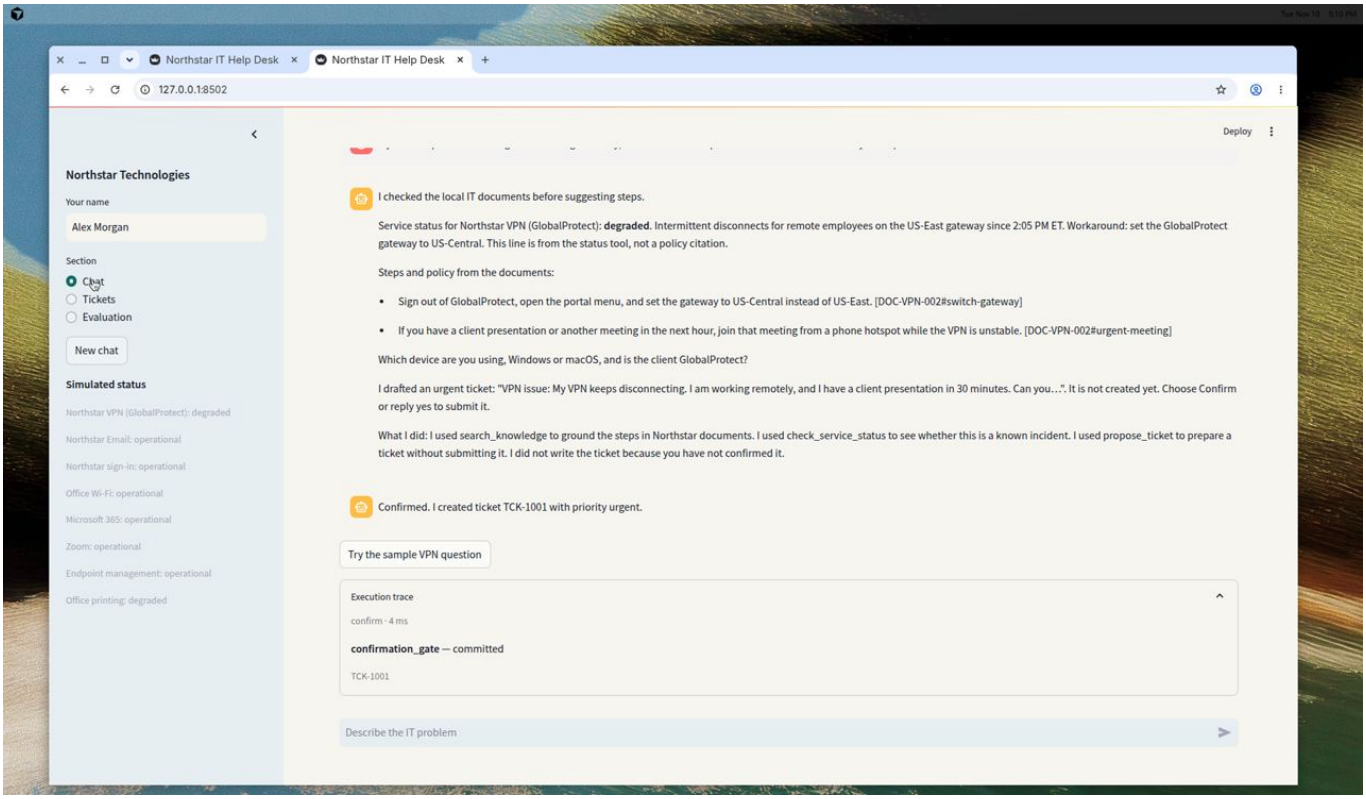
BANNER ID: _____

Vibe Coding vs. Spec-Driven Coding

Demo 1: Vibe Coding

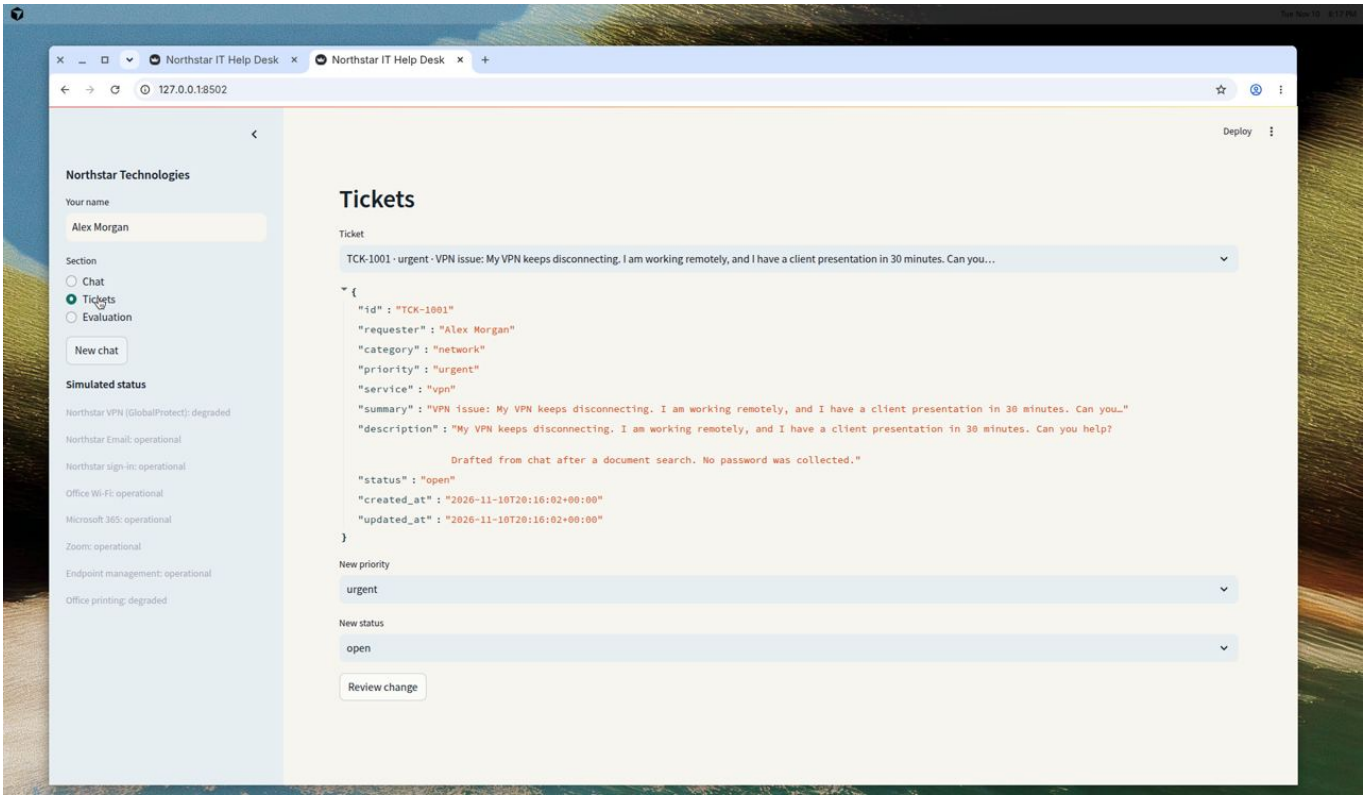
Screenshot 1: AI IT Help Desk - Chat and RAG

Shows VPN troubleshooting, document citations, a clarifying question, and the actions the assistant took.



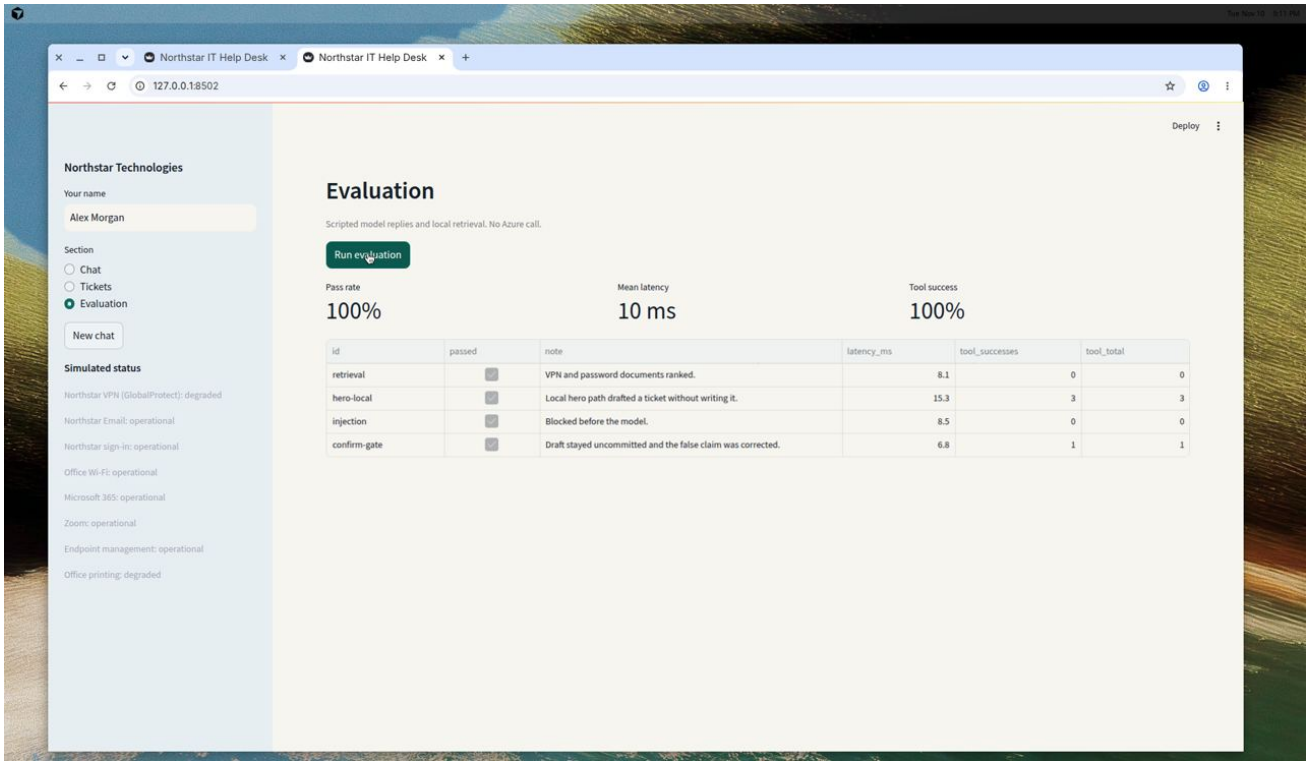
Screenshot 2: Support ticket creation

Shows ticket TCK-1001 stored in SQLite only after the employee confirmed the draft.



Screenshot 3: Evaluation dashboard

Shows 4 of 4 cases passed, a 100 percent pass rate, mean latency, and tool success.



I built the Northstar Technologies AI IT Help Desk with Python and Streamlit by working directly in one program, demo1_vibe/helpdesk_app.py. I created a local virtual environment, installed the required libraries, and added the pieces the demo needed: an interactive chat, TF-IDF search over fictional IT documents, a simulated service-status check, a ticket draft, and a confirmation step before SQLite stores the ticket.

The sample employee says the VPN keeps disconnecting before a client presentation. The reply cites Northstar documents, including the step to set the GlobalProtect gateway to US-Central, reports that VPN status is degraded, asks whether the laptop is Windows or macOS, and explains which tools ran and why. The execution trace lists those stages. It does not show private model reasoning. A message that tries to override the instructions is blocked before any tool runs.

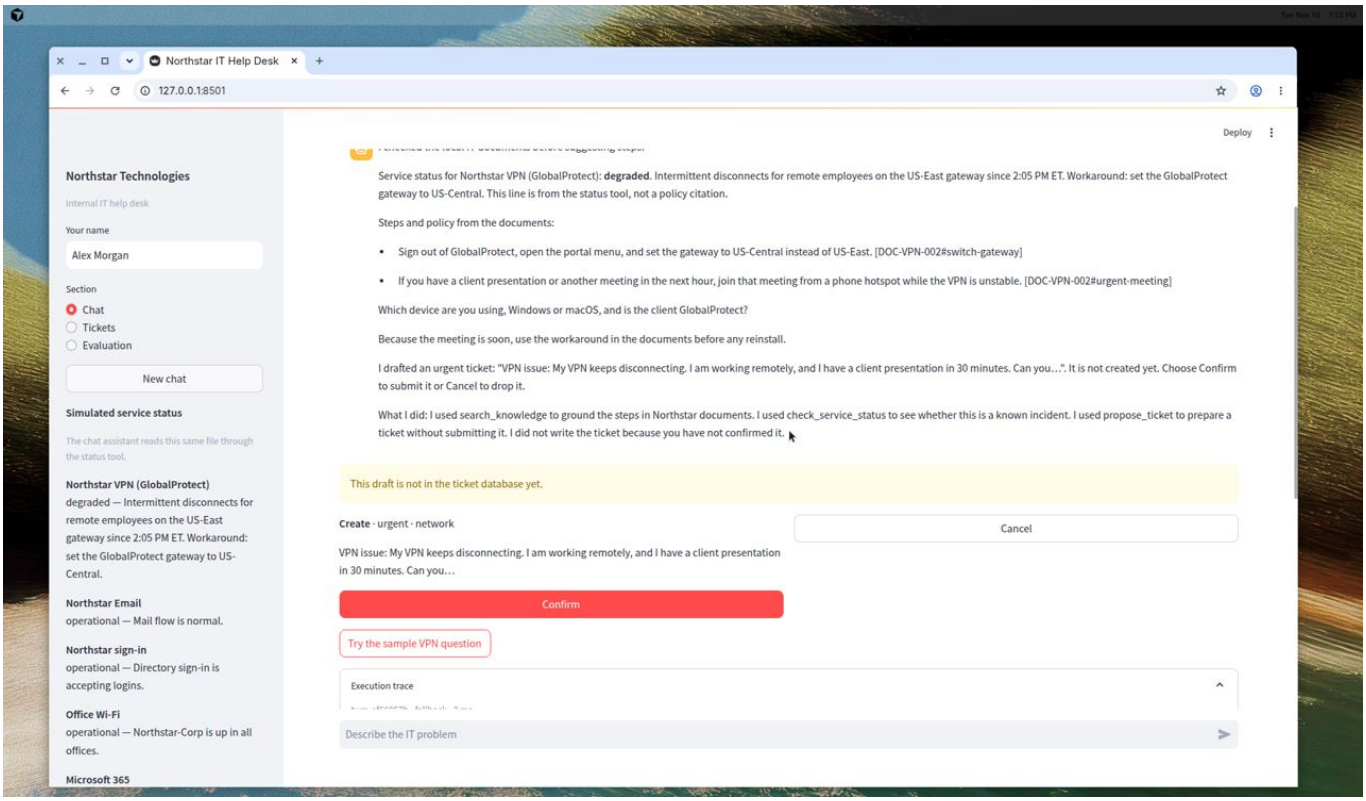
Azure Claude is read only from a local .env file. No API key is stored in the repository. The demo and the automated tests use the local answer path. pytest collected 6 tests for this demo, and all 6 passed. The evaluation page reported 4 of 4 cases passed, a mean latency of 10 ms, and 100 percent tool success.

This demo showed how an assisted coding session can produce a working help desk in one file, while still keeping credentials out of the source and refusing to save a ticket until the employee confirms it.

Demo 2: Spec-Driven Coding

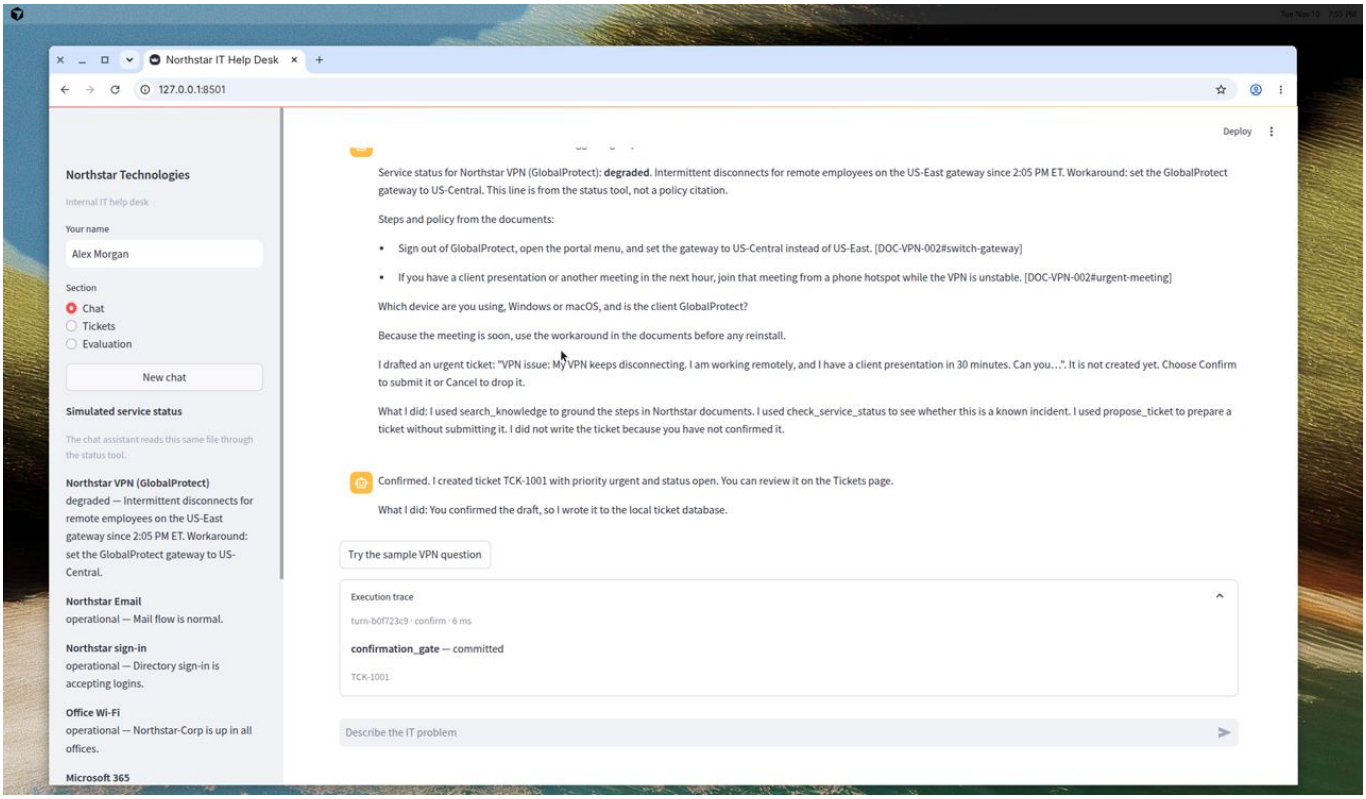
Screenshot 1: AI IT Help Desk - Chat and RAG

Shows cited troubleshooting steps and a ticket draft that has not been written to the database.



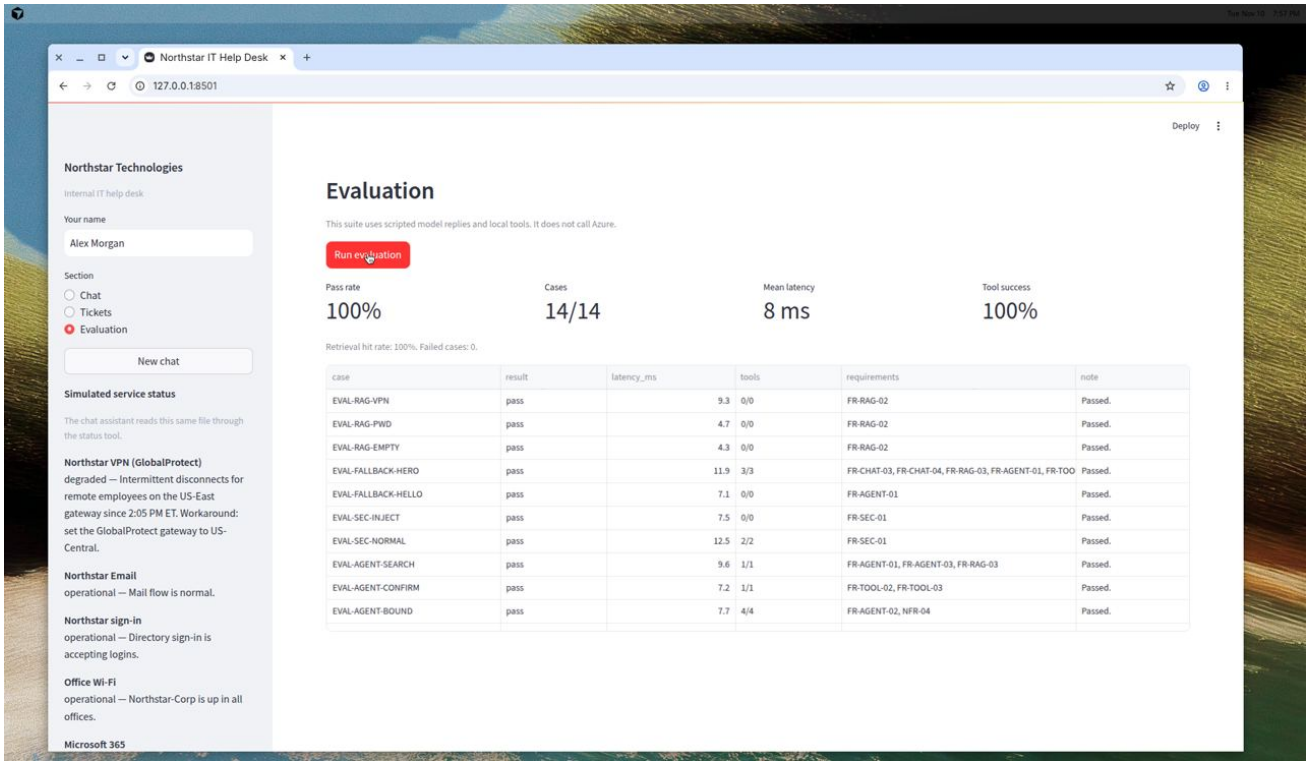
Screenshot 2: Support ticket confirmation

Shows that ticket TCK-1001 is created only after the employee clicks Confirm.



Screenshot 3: Evaluation dashboard

Shows 14 of 14 cases passed, a 100 percent pass rate, mean latency, and 100 percent tool success.



I built the same help desk a second time. This version started from written specifications. It does not extend or import the Demo 1 program.

Phase 1 - Generate the specification

Before writing application code, I created six documents in the specs folder: requirements.md, architecture.md, agent-design.md, security.md, evaluation.md, and implementation-plan.md. Every feature has an identifier, the expected behavior, acceptance criteria, test cases, and error handling.

Phase 2 - Review and approve

I reviewed the specifications for missing pieces, contradictions, and unnecessary complexity. The review separates steps that should stay ordinary Python from steps that need the model. It also explains how citations are checked against the passages that were retrieved, how an unknown tool is rejected, how a ticket write waits for an explicit confirmation, how pass rate, latency, and tool success are measured, and how a model or tool failure is handled. The requirements-to-tests matrix is in specs/review.md.

Phase 3 - Implement against the specification

After that review, I implemented demo2_spec as separate parts for retrieval, tickets, tools, security, and the model client. The model client does not decide when a ticket may be saved. Automated tests use a mock model, so they do not call Azure. pytest collected 34 tests for this demo, and all 34 passed. The evaluation dashboard ran 14 scripted cases: 14 passed, mean latency 8 ms, and tool success 100 percent.

The screenshots show the cited VPN steps, a draft that stays out of SQLite until Confirm is clicked, and the evaluation results. Writing the specification first made each security rule and each test traceable to a requirement. Demo 1 reached a working program in one file. Demo 2 is the build I would continue, because the behavior, the tests, and the documents stay aligned.